INT104 Artificial Intelligence
Lecture 8: Non-parametric Learning

Dr. Shengchen Li

Department of Intelligent Science
Xi'an Jiaotong-Liverpool University

Outline

- Non-parametric modeling: motivation and characteristics
- k-Nearest Neighbour (kNN): idea, pros/cons, exercise
- Decision trees: basic procedure and feature selection metrics
- Hierarchical (agglomerative) clustering
- k-means clustering and hyper-parameter selection

Non-parametric Modeling

- Non-parametric models can be used with arbitrary distributions.
- No need to assume a known parametric form for the underlying density.
- Can handle **multimodal** distributions (common in practice).

Intuition: model complexity can grow with data, instead of being fixed by a small set of parameters.

Pros and Cons (Non-parametric Methods)

Pros

- No assumptions needed about distributions ahead of time (generality).
- With enough samples, can converge to arbitrarily complicated target densities.

Cons

- Sample complexity may be very large (often grows exponentially with dimensionality).
- Sensitive to window size / neighborhood size:
  - Too small $\Rightarrow$ most volume empty (high variance).
  - Too large $\Rightarrow$ important variations lost (high bias).
- Potentially severe computation time and storage requirements.

k-Nearest Neighbour (kNN): Core Idea

- Training set: data points with known class labels.
- For a new data point x:
  1. Compute similarity/distance from x to each training sample.
  2. Select the k most similar samples (the k **nearest neighbours**).
  3. Predict label by **majority vote** among neighbours' labels.

kNN: Adaptive Window View

- Goal: solution when the best window function is unknown.
- Let the cell (neighborhood) volume depend on the training data.
- Center a cell around x and grow it until it contains k_n samples.
- Those k_n samples are the nearest neighbours of x.

kNN Exercise (Play Tennis)

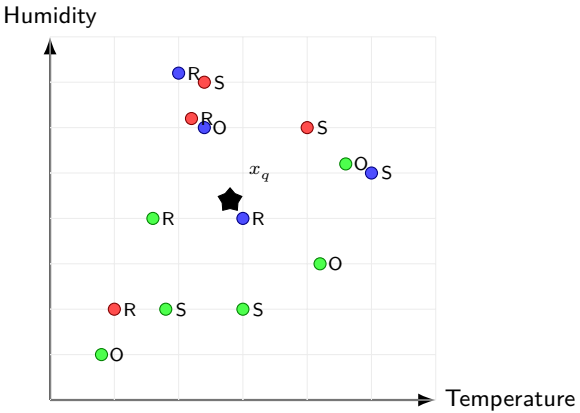
Outlook	Temperature	Humidity	Windy	Play
sunny	85	85	1.0	maybe
sunny	80	90	3.0	no
overcast	83	86	0.8	yes
rainy	70	96	0.2	maybe
rainy	68	80	0.1	yes
rainy	65	70	2.8	no
overcast	64	65	2.6	yes
sunny	72	95	0.3	no
sunny	69	70	0.5	yes
rainy	75	80	0.4	maybe
sunny	75	70	2.2	yes
overcast	72	90	2.4	maybe
overcast	81	75	0.0	yes
rainy	71	91	2.9	no

Query: Will we play in the case of **(sunny, 74, 82)**?

In-class Discussion (kNN)

- What decision you made that affects the prediction?
- Typical key choices:
  - Distance metric (e.g., Euclidean, Manhattan, mixed categorical + numerical)
  - Feature scaling / normalization
  - Handling categorical features (e.g., one-hot encoding)
  - Choice of k
  - Weighted vs unweighted vote

Diagram: Temperature vs Humidity (Outlook as Marker, Label as Color)

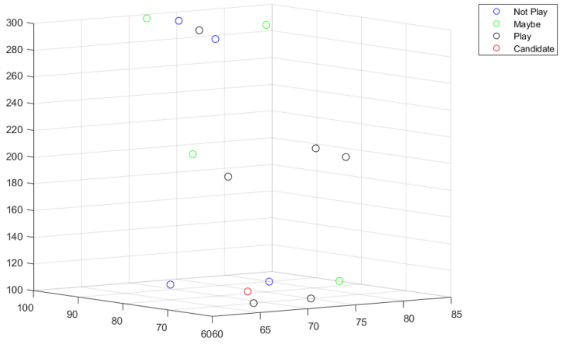


Legend

- Color = label: Yes, No, Maybe
- Marker text: S=`sunny`, O=`overcast`, R=`rainy`
- Black star = query $(74, 82)$ with Outlook=`sunny`

kNN idea: find the k closest points to the star using the mixed distance.

Different Weights for Outlook



Find the k Nearest Neighbours (Example with $k = 3$)

For this query, the nearest neighbours (by the distance above) are:

Rank	Outlook	Temp	Humidity	Play
1	sunny	75	70	yes
2	sunny	85	85	maybe
3	sunny	80	90	no

Majority vote among 3-NN:

$$\{\texttt{yes},\ \texttt{maybe},\ \texttt{no}\} \Rightarrow \text{tie } (1\text{–}1\text{–}1)$$

Tie-breaking options (choose one policy):

- Use distance-weighted voting (closer neighbour counts more)
- Choose a different k (e.g., $k = 1$)

What Decisions Affect the Prediction?

In this example, the prediction depends on:

- How we encode the categorical feature (Outlook) and its penalty weight
- Feature scaling for Temperature/Humidity
- Choice of k
- Tie-breaking rule (especially important with multi-class labels: yes/no/maybe)
- Whether we use weighted or unweighted voting

Decision Tree: Basic Procedure

- Training dataset: $D = \{x_1, x_2, ..., x_n\}$ with labels $Y = \{y_1, ..., y_n\}$,

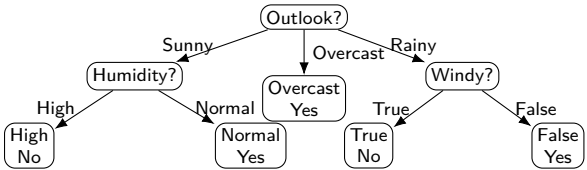
$$x_i = (x_{i1}, x_{i2}, ..., x_{im})$$

- Given a node (root node), determine whether it is a leaf by checking:
  - The node contains no samples
  - All samples in the node belong to a single class
  - A common attribute is shared / no useful split
  - Attributes remain to be further analyzed
- Leaf decision is often determined by **voting** (majority class).
- Select an attribute x_j to build a branch for each attribute value and recurse.

Decision Tree Example: Play Tennis

Rule sketch

- Root: Outlook (Sunny / Overcast / Rainy)
- If Overcast: **Yes**
- If Sunny: check Humidity
  - High → No
  - Normal → Yes
- If Rainy: check Windy
  - True → No
  - False → Yes



Selecting the Feature to Split

- How could we use metrics to select the feature to be analyzed?
- Goal: choose the feature that is most distinguishable / best performance.
- Common selection metrics:
  - Entropy / Information Gain
  - Gain Ratio
  - Gini Index

Entropy and Information Gain

Entropy

$$\text{entropy}(D) = -\sum_{k=1}^{|Y|} p_k \log_2 p_k$$

Information Gain

$$\text{Gain}(D, x_j^*) = \text{entropy}(D) - \sum_{v=1}^{V} \frac{|D_v|}{|D|} \text{entropy}(D^{(v)})$$

- V: number of values for attribute x_j^*
- D_v: subset where attribute takes value v

Gain Ratio (Selection Metric)

Gain Ratio

$$\mathsf{GainRatio}(D, x_j^*) = \frac{\mathsf{Gain}(D, x_j^*)}{\mathsf{IV}(a)}$$

Intrinsic Value

$$\mathsf{IV}(a) = -\sum_{v=1}^{V} \frac{|D_v|}{|D|} \log_2 \left(\frac{|D_v|}{|D|} \right)$$

Gini Index (Selection Metric)

Gini Index idea (impurity measure)

- Often used in CART-style decision trees.
- Lower impurity after split is better.

A common form:

$$\text{Gini}(D) = 1 - \sum_{k=1}^{|Y|} p_k^2$$

and choose splits that reduce impurity the most (weighted by subset sizes).

Hierarchical Clustering (Agglomerative)

- General outline:
  1. For n objects $v_1, \ldots, v_n$, assign each to a singleton cluster $C_i = \{v_i\}$.
  2. Define cluster similarity / distance measure (e.g., Euclidean, cosine).
  3. Repeat until one cluster remains:
     - Identify the two most similar clusters (ties possible).
     - Merge them: remove C_j, C_k and add $C_j \cup C_k$.
  4. Use a **dendrogram** to visualize merge sequence.
- Distance between clusters $D(C_i, C_j)$ can be defined in multiple ways:
  - **Single linkage**: minimum distance between any pair of points across clusters.
  - **Complete linkage**: maximum distance between any pair of points across clusters.
  - Others: average linkage, median linkage, etc.

Demonstration Dataset (Distance Matrix)

We will use 5 objects: A, B, C, D, E with distances:

	A	B	C	D	E
A	0				
B	8	0			
C	3	6	0		
D	5	5	8	0	
E	13	10	2	7	0

We will demonstrate **single linkage** first.

Step 0: Initialize Clusters

Start with singleton clusters:

$$\{A\}, \{B\}, \{C\}, \{D\}, \{E\}$$

Find the smallest distance in the matrix (excluding zeros on diagonal):

$$\min d(i, j) = d(C, E) = 2$$

Step 1: Merge the Closest Pair

Merge: $\{C\}$ and $\{E\}$ at distance 2.

New cluster:

$$C_1 = \{C, E\}$$

Remaining clusters:

$$\{A\}, \{B\}, \{D\}, \{C, E\}$$

Let's use F to donate the new mini-cluster.

Update Distances (Single Linkage)

For single linkage, distance from new cluster to another cluster is the minimum pairwise distance.

Compute:

$$D(F, A) = \min(d(C, A), d(E, A)) = \min(3, 13) = 3$$

$$D(F, B) = \min(d(C, B), d(E, B)) = \min(6, 10) = 6$$

$$D(F, C) = \min(d(C, D), d(E, D)) = \min(8, 7) = 7$$

Step 2: Next Merge

Now consider distances among clusters:

	A	B	D	F
A	0			
B	8	0		
D	5	5	0	
F	3	6	7	0

Smallest is 3:

Merge $\{A\}$ with $\{F\}$ at distance 3

i.e., New cluster:

$$G = \{A, C, E\}$$

Update Distances After Step 2 (Single Linkage)

Compute distances from $G = \{A, C, E\}$ to the remaining singletons $\{B\}, \{D\}$:

$$D(G, \{B\}) = \min(d(A, B), d(C, B), d(E, B)) = \min(8, 6, 10) = 6$$
$$D(G, \{D\}) = \min(d(A, D), d(C, D), d(E, D)) = \min(5, 8, 7) = 5$$

Remaining clusters:

$$\{B\}, \{D\}, \{A, C, E\}$$

Step 3: Next Merge

Current inter-cluster distances:

$$d(B, D) = 5, \quad D(G, D) = 5, \quad D(G, B) = 6$$

	B	D	G
B	0		
D	5	0	
G	6	5	0

Option (tie-break): merge $\{\mathrm{B}\}$ and $\{\mathrm{D}\}$ at distance 5.

$$H = \{B, D\}$$

Step 4: Final Merge

Now only two clusters remain:

$$G = \{A, C, E\}, \quad H = \{B, D\}$$

Single linkage distance:

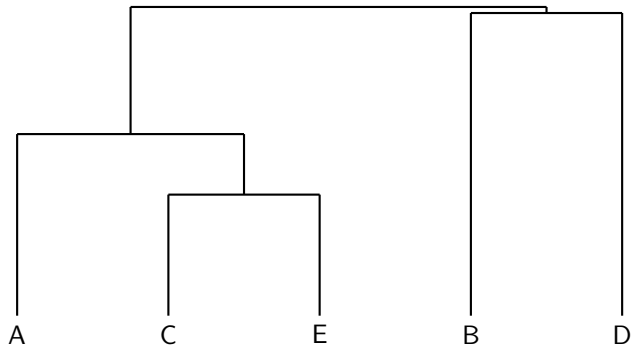
$$D(G, H) = \min\{d(A, B), d(A, D), d(C, B), d(C, D), d(E, B), d(E, D)\}$$

$$= \min\{8, 5, 6, 8, 10, 7\} = 5$$

Final merge at distance 5.

Dendrogram (Single Linkage, One Valid Tie-break)

Below is a schematic dendrogram (not to exact scale), showing the merge sequence:



Note: because of ties, multiple dendrograms can be correct depending on merge order.

What Changes with Complete / Average Linkage?

- **Single linkage** tends to create *chains* (can link clusters via one close pair).
- **Complete linkage** prefers compact clusters (depends on farthest pair).
- **Average linkage** is a compromise (uses mean distance across pairs).

Same data + different linkage $\Rightarrow$ potentially different merge order and dendrogram.

Algorithm Summary (Agglomerative Clustering)

1. Initialize clusters as singletons.
2. Choose linkage rule to compute $D(C_a, C_b)$.
3. Repeat:
   - Find pair (C_a, C_b) with minimal $D(C_a, C_b)$
   - Merge them, record the merge distance (height in dendrogram)
   - Update distances between the new cluster and all others
4. Stop when one cluster remains.

How to Read a Dendrogram

- Leaves: individual data points.
- Each merge: a horizontal line at height equal to merge distance.
- Cutting the dendrogram at a chosen height gives a clustering.
- The chosen cut height corresponds to an implicit number of clusters.

k-means: Algorithm

- k-means: group objects into k clusters based on feature similarity.
- Choose k (positive integer) and initialize centroids.
- Iterate until convergence:
  1. Assign each sample to the cluster with the nearest centroid.
  2. Recompute centroid of each cluster.
- Stop when no samples change cluster assignment after an iteration.

Example of k-means (2 Clusters)

Given the 7 points:

$$(1.0, 1.0),\ (1.5, 2.0),\ (3.0, 4.0),\ (5.0, 7.0),\ (3.5, 5.0),\ (4.5, 5.0),\ (3.5, 4.5)$$

- Use k-means to divide all points into two clusters ($k = 2$).
- Show:
  - initialization (chosen centroids),
  - assignment step,
  - centroid update step,
  - iterations until convergence.

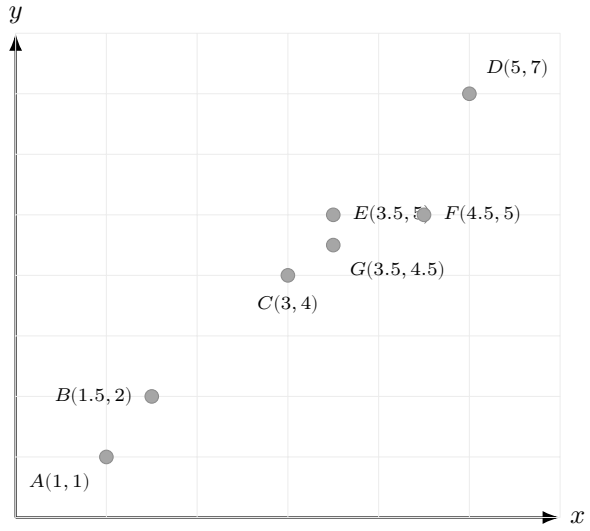
k-means Procedure (with City-block Distance)

Repeat until convergence:

1. **Initialize** 2 centroids μ_1, μ_2
2. **Assign** each point to nearest centroid by d_1
3. **Update** centroids as the mean of assigned points

Note: Standard k-means updates use the mean (works best with Euclidean). Here we keep the *standard k-means update* (mean) but use *city-block* distance for assignment, as requested.

Step 0: Plot the Points



Initialization

Choose initial centroids (common simple choice: pick two points):

$$\mu_1^{(0)} = (1, 1) \quad \text{(use point } A\text{)}, \qquad \mu_2^{(0)} = (5, 7) \quad \text{(use point } D\text{)}.$$

We will now assign each point to the nearer centroid using city-block distance.

Iteration 1: Assignment (City-block Distances)

Compute $d_1(\text{point}, \mu_1^{(0)})$ and $d_1(\text{point}, \mu_2^{(0)})$.

Point	$d_1(\cdot, \mu_1^{(0)}{=}(1,1))$	$d_1(\cdot, \mu_2^{(0)}{=}(5,7))$	Cluster
$A(1,1)$	0	$\lvert 1-5\rvert + \lvert 1-7\rvert = 10$	1
$B(1.5,2)$	$\lvert 0.5\rvert + \lvert 1\rvert = 1.5$	$\lvert -3.5\rvert + \lvert -5\rvert = 8.5$	1
$C(3,4)$	$\lvert 2\rvert + \lvert 3\rvert = 5$	$\lvert -2\rvert + \lvert -3\rvert = 5$	tie
$D(5,7)$	10	0	2
$E(3.5,5)$	$\lvert 2.5\rvert + \lvert 4\rvert = 6.5$	$\lvert -1.5\rvert + \lvert -2\rvert = 3.5$	2
$F(4.5,5)$	$\lvert 3.5\rvert + \lvert 4\rvert = 7.5$	$\lvert -0.5\rvert + \lvert -2\rvert = 2.5$	2
$G(3.5,4.5)$	$\lvert 2.5\rvert + \lvert 3.5\rvert = 6$	$\lvert -1.5\rvert + \lvert -2.5\rvert = 4$	2

Tie-breaking (for C): assign ties to Cluster 1 (any fixed rule is acceptable).

Iteration 1: Clusters After Assignment

With the tie-breaking above:

$$\mathcal{C}_1^{(1)} = \{A, B, C\}, \qquad \mathcal{C}_2^{(1)} = \{D, E, F, G\}.$$

Now update centroids by the (coordinate-wise) mean:

$$\mu_j^{(1)} = \frac{1}{|\mathcal{C}_j^{(1)}|} \sum_{x_i \in \mathcal{C}_j^{(1)}} x_i.$$

Iteration 1: Update Centroids (Means)

Cluster 1:

$$\mu_1^{(1)} = \frac{1}{3}[(1,1) + (1.5,2) + (3,4)] = \left(\frac{5.5}{3}, \frac{7}{3}\right) \approx (1.833,\ 2.333).$$

Cluster 2:

$$\mu_2^{(1)} = \frac{1}{4}[(5,7) + (3.5,5) + (4.5,5) + (3.5,4.5)] = \left(\frac{16.5}{4}, \frac{21.5}{4}\right) = (4.125,\ 5.375).$$

Iteration 2: Re-assignment Using City-block Distance

Now compute distances to new centroids:

$$\mu_1^{(1)} \approx (1.833, 2.333), \quad \mu_2^{(1)} = (4.125, 5.375).$$

Point	$d_1(\cdot, \mu_1^{(1)})$	$d_1(\cdot, \mu_2^{(1)})$	Cluster
$A(1,1)$	$\lvert-0.833\rvert + \lvert-1.333\rvert = 2.166$	$\lvert-3.125\rvert + \lvert-4.375\rvert = 7.5$	1
$B(1.5,2)$	$\lvert-0.333\rvert + \lvert-0.333\rvert = 0.666$	$\lvert-2.625\rvert + \lvert-3.375\rvert = 6.0$	1
$C(3,4)$	$\lvert1.167\rvert + \lvert1.667\rvert = 2.834$	$\lvert-1.125\rvert + \lvert-1.375\rvert = 2.5$	2
$D(5,7)$	$\lvert3.167\rvert + \lvert4.667\rvert = 7.834$	$\lvert0.875\rvert + \lvert1.625\rvert = 2.5$	2
$E(3.5,5)$	$\lvert1.667\rvert + \lvert2.667\rvert = 4.334$	$\lvert-0.625\rvert + \lvert-0.375\rvert = 1.0$	2
$F(4.5,5)$	$\lvert2.667\rvert + \lvert2.667\rvert = 5.334$	$\lvert0.375\rvert + \lvert-0.375\rvert = 0.75$	2
$G(3.5,4.5)$	$\lvert1.667\rvert + \lvert2.167\rvert = 3.834$	$\lvert-0.625\rvert + \lvert-0.875\rvert = 1.5$	2

Only C moved (from Cluster 1 to Cluster 2). Continue to update centroids.

Iteration 2: Update Centroids

Now:

$$\mathcal{C}_1^{(2)} = \{A, B\}, \qquad \mathcal{C}_2^{(2)} = \{C, D, E, F, G\}.$$

Update means:

$$\mu_1^{(2)} = \frac{(1,1) + (1.5,2)}{2} = (1.25, 1.5).$$

$$\mu_2^{(2)} = \frac{(3,4) + (5,7) + (3.5,5) + (4.5,5) + (3.5,4.5)}{5} = (3.9, 5.1).$$

Iteration 3: Check Assignments (Convergence)

Centroids:

$$\mu_1^{(2)} = (1.25, 1.5), \quad \mu_2^{(2)} = (3.9, 5.1).$$

Point	$d_1(\cdot, \mu_1^{(2)})$	$d_1(\cdot, \mu_2^{(2)})$	Cluster
$A(1,1)$	$0.25 + 0.5 = 0.75$	$2.9 + 4.1 = 7.0$	1
$B(1.5,2)$	$0.25 + 0.5 = 0.75$	$2.4 + 3.1 = 5.5$	1
$C(3,4)$	$1.75 + 2.5 = 4.25$	$0.9 + 1.1 = 2.0$	2
$D(5,7)$	$3.75 + 5.5 = 9.25$	$1.1 + 1.9 = 3.0$	2
$E(3.5,5)$	$2.25 + 3.5 = 5.75$	$0.4 + 0.1 = 0.5$	2
$F(4.5,5)$	$3.25 + 3.5 = 6.75$	$0.6 + 0.1 = 0.7$	2
$G(3.5,4.5)$	$2.25 + 3.0 = 5.25$	$0.4 + 0.6 = 1.0$	2

Assignments did not change $\Rightarrow$ **converged**.

Final Result (k=2)

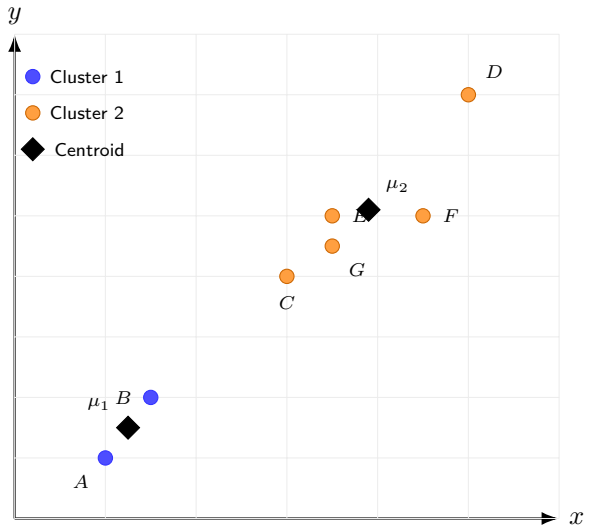
Cluster 1:

$$\{(1,1),\ (1.5,2)\} \quad \text{with centroid } \mu_1 = (1.25, 1.5).$$

Cluster 2:

$$\{(3,4),\ (5,7),\ (3.5,5),\ (4.5,5),\ (3.5,4.5)\} \quad \text{with centroid } \mu_2 = (3.9, 5.1).$$

Diagram: Final Clusters (City-block Assignment)



Results and Hyper-parameters

- Determine hyper-parameters:
  - Try different k values: what results do we get?
  - Can we decide the best k for best performance?
- Typical approaches (conceptually):
  - Elbow method (within-cluster SSE vs k)
  - Silhouette score
  - Validation on downstream task (if clustering is a preprocessing step)

Summary

- Non-parametric methods: flexible, data-driven complexity; trade-offs in sample/computation.
- kNN: local voting; sensitive to k and distance/feature scaling.
- Decision trees: recursive partitioning; split selection via entropy/gain, gain ratio, gini.
- Clustering: hierarchical (linkage choices) and k-means (choose k).